

SIMATS ENGINEERING
CO2 ASSESSMENT TOOL 3 DEBUGGING & OPTIMISATION TASK

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Name : C.Himaja

Reg.No : 192373035

Q11. Debugging Selection Sort (Incorrect Swap Position)

The given Selection Sort implementation incorrectly swaps elements even when the minimum index is unchanged, leading to unnecessary operations. Identify the logical error in

the swapping mechanism and explain its impact. Debug the code step-by-step to correct the

swapping condition. Optimize the implementation to reduce unnecessary swaps. Analyze the

time complexity of the corrected version and rewrite the optimized code with proper comments.

```
def selection_sort(arr):  
    for i in range(len(arr)):  
        min_idx = i  
        for j in range(i+1, len(arr)):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ISSUE  
    return arr
```

1. Source Code (C Program)

```
#include <stdio.h>

// Function to perform optimized Selection Sort
void selectionSort(int arr[], int n)
{
    int i, j, minIndex, temp;

    // Traverse the array
    for(i = 0; i < n - 1; i++)
    {
        minIndex = i;

        // Find the index of the minimum element
        for(j = i + 1; j < n; j++)
        {
            if(arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        // Swap only if the minimum element is not already in the correct position
        if(minIndex != i)
        {
            temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```

```

}

// Function to print array
void printArray(int arr[], int n)
{
    int i;
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

int main()
{
    int n, i;

    printf("Enter number of transaction amounts: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter transaction amounts:\n");

    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("\nOriginal Array:\n");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("\n\nSorted Array:\n");

```

```
    printArray(arr, n);

    return 0;
}
```

Sample Input

Enter number of transaction amounts: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

Sample Output

Original Array:

4500 1200 3800 2500 5100 1900

Sorted Array:

1200 1900 2500 3800 4500 5100

Time Complexity

Case	Complexity
------	------------

Best Case	$O(n^2)$
-----------	----------

Average Case	$O(n^2)$
--------------	----------

Worst Case	$O(n^2)$
------------	----------

Space Complexity

$O(1)$

Result

The corrected Selection Sort successfully sorts the transaction amounts while avoiding unnecessary swap operations.